Terrain-Aware Planner: Concept, Optimization, and Trajectory Equations

NEMo / Neural Elevation Models

May 21, 2026

1 Purpose and Concept

The terrain-aware planner in `nemo/terrain_aware_planner.py` optimizes a 2D path over a learned NEMo height field. The terrain is modeled as a differentiable surface

$$z = h(x, y),$$

where (x, y) are dataset-frame horizontal coordinates and z is terrain elevation. The planner does not optimize a free 3D curve. Instead, it optimizes horizontal control points, samples a smooth path through those controls, then asks the height field for the corresponding terrain elevation and local gradient at every sampled point.

The planner is intended to convert an initial seed path, usually from A* or hand-selected waypoints, into a path that is more feasible for a rover or low-altitude terrain follower. Feasibility is represented by differentiable proxy costs for:

- total 3D path length,
- uphill slope,
- downhill slope,
- cross-track slope or roll demand,
- throttle demand from gravity along the path,
- steering demand from curvature,
- path and control-point smoothness.

The endpoint positions are fixed. Only interior control points move. This preserves the requested start and goal while allowing the path to bend around terrain hazards such as steep crater walls or hills.

2 Height Field Interaction

The planner interacts with NEMo through `nemo.field.h(xy)` and `nemo.field.bounds`. For a sampled path

$$\mathbf{p}_i = (x_i, y_i),$$

the terrain elevation is

$$z_i = h(x_i, y_i).$$

The planner can compute height gradients in two modes. The default mode is `gradient_mode="finite_differenc` exposed in the crater-entry driver as `--gradients fd`. It computes central finite differences:

$$h_x(x_i, y_i) \approx \frac{h(x_i + \epsilon, y_i) - h(x_i - \epsilon, y_i)}{2\epsilon}, \qquad h_y(x_i, y_i) \approx \frac{h(x_i, y_i + \epsilon) - h(x_i, y_i - \epsilon)}{2\epsilon}.$$

If `finite_difference_eps` is not set, the code chooses

$$\epsilon = \max\left(\frac{\min(x_{\max} - x_{\min},\, y_{\max} - y_{\min})}{512},\; 10^{-3}\right).$$

The alternative mode is `gradient_mode="autograd"`, exposed as `--gradients autograd`. In that mode, the planner computes

$$\nabla h_i = \frac{\partial h(x_i, y_i)}{\partial (x_i, y_i)}$$

with `torch.autograd.grad`. The implementation keeps the query tensor connected to the sampled path so the final objective can still backpropagate to the B-spline control points.

The gradient vector is

$$\nabla h_i = [h_x(x_i, y_i),\; h_y(x_i, y_i)].$$

This gradient is the key link between the learned surface and the vehicle-aware costs: it defines terrain slope magnitude, slope along the path direction, and lateral slope across the path.

3 Path Parameterization

The optimizer does not directly move every sampled point. It optimizes a compact set of B-spline control points:

$$\mathbf{c}_0, \mathbf{c}_1, \ldots, \mathbf{c}_{M-1}.$$

The first and last controls are fixed to the seed start and goal:

$$\mathbf{c}_0 = \mathbf{p}_{start}, \qquad \mathbf{c}_{M-1} = \mathbf{p}_{goal}.$$

The interior controls are trainable:

$$\theta = \{\mathbf{c}_1, \ldots, \mathbf{c}_{M-2}\}.$$

A clamped B-spline basis B_{ij} samples N path points from those controls:

$$\mathbf{p}_i = \sum_{j=0}^{M-1} B_{ij}\mathbf{c}_j.$$

The implementation explicitly overwrites the first and last sampled points with the first and last controls, guaranteeing endpoint interpolation even after numerical basis normalization.

4 Differentiable Trajectory Quantities

For every sampled point $\mathbf{p}_i = (x_i, y_i)$, the planner builds a differentiable trajectory dictionary. The derivatives below are discrete derivatives with respect to sample index in the implementation.

4.1 Tangent, Lateral Axis, and Yaw

Let

$$\dot{\mathbf{p}}_i = [\dot{x}_i, \dot{y}_i]$$

be the first discrete derivative of the sampled path. The planar speed in sample-index units is

$$q_i = \|\dot{\mathbf{p}}_i\|_2.$$

The unit tangent and left-lateral axes are

$$\mathbf{t}_i = \frac{\dot{\mathbf{p}}_i}{\max(q_i, 10^{-8})}, \qquad \mathbf{l}_i = [-t_{iy},\ t_{ix}].$$

The yaw angle is

$$\psi_i = \operatorname{atan2}(\dot{y}_i, \dot{x}_i).$$

4.2 Curvature and Steering

Let the second discrete derivative be

$$\ddot{\mathbf{p}}_i = [\ddot{x}_i, \ddot{y}_i].$$

The planar curvature is

$$\kappa_i = \frac{\dot{x}_i\ddot{y}_i - \dot{y}_i\ddot{x}_i}{(\dot{x}_i^2 + \dot{y}_i^2)^{3/2}}.$$

The steering proxy uses a bicycle-model approximation with wheelbase L:

$$\delta_i = \arctan(L\kappa_i).$$

4.3 Slope Magnitude

The terrain slope magnitude in radians is

$$\alpha_i = \arctan(\|\nabla h_i\|_2).$$

This is reported in diagnostics as degrees:

$$\alpha_{deg,i} = \frac{180}{\pi}\alpha_i.$$

4.4 Along-Track and Cross-Track Slope

The grade along the vehicle forward direction is

$$g_i^{along} = \nabla h_i \cdot \mathbf{t}_i.$$

The grade toward the left lateral axis is

$$g_i^{cross} = \nabla h_i \cdot \mathbf{l}_i.$$

The planner also stores angle forms:

$$\beta_i^{along} = \arctan(g_i^{along}), \qquad \beta_i^{cross} = \arctan(g_i^{cross}).$$

Positive β^{along} means uphill motion along the direction of travel. Negative β^{along} means downhill motion. The magnitude of β^{cross} is a roll-demand proxy.

4.5 Throttle Proxy

The planner uses a simplified gravity-along-track acceleration proxy:

$$a_i^{req} = g\, g_i^{along},$$

where g is the configured gravity. It then normalizes by the configured maximum throttle acceleration $a_{\max}$:

$$u_i^{thr} = \frac{a_i^{req}}{a_{\max}}.$$

This is not a full vehicle dynamics model. It is a differentiable proxy that penalizes routes requiring large acceleration to climb or control descent.

4.6 3D Length

The sampled terrain-following 3D segment length is

$$\Delta s_i = \sqrt{(x_{i+1} - x_i)^2 + (y_{i+1} - y_i)^2 + (z_{i+1} - z_i)^2}.$$

The path length term is

$$L_{3D} = \sum_{i=0}^{N-2} \Delta s_i.$$

5 Objective Function

The raw objective terms are computed in `compute_raw_costs`. With normalization enabled, each term is divided by its initial-path value before weighting. The total objective is:

$$\begin{aligned} J = w_L \frac{J_L}{\bar{J}_L} + w_U \frac{J_U}{\bar{J}_U} + w_D \frac{J_D}{\bar{J}_D} + w_C \frac{J_C}{\bar{J}_C} \\ + w_T \frac{J_T}{\bar{J}_T} + w_S \frac{J_S}{\bar{J}_S} + w_R \frac{J_R}{\bar{J}_R}. \end{aligned}$$

Here $\bar{J}$ values are initial-path normalizers when `normalize_costs=True`; otherwise they are 1.

5.1 Length Cost

$$J_L = L_{3D}.$$

5.2 Directional Slope Costs

The uphill angle is

$$u_i = \max(\beta_i^{along}, 0).$$

The downhill angle is

$$d_i = \max(-\beta_i^{along}, 0).$$

The cross-track angle is

$$c_i = |\beta_i^{cross}|.$$

For a directional angle r_i and limit $r_{\max}$, the planner uses:

$$J(r, r_{\max}) = \text{mean}\left[\left(\frac{r_i}{r_{\max}}\right)^2\right] + \text{mean}\left[\left(\frac{\max(r_i - r_{\max}, 0)}{r_{\max}}\right)^2\right].$$

Therefore:

$$J_U = J(u, \beta_{\max}^{up}), \qquad J_D = J(d, \beta_{\max}^{down}), \qquad J_C = J(c, \beta_{\max}^{cross}).$$

The first part discourages slope even below the limit. The second part adds an extra soft violation penalty after the limit.

5.3 Throttle Cost

$$J_T = \text{mean}\left[(u_i^{thr})^2\right] + \text{mean}\left[\max(|u_i^{thr}| - 1, 0)^2\right].$$

The first term penalizes throttle demand generally. The second term penalizes values beyond the nominal normalized limit.

5.4 Steering Cost

$$J_S = \text{mean}\left[\left(\frac{\delta_i}{\delta_{\max}}\right)^2\right] + \text{mean}\left[\left(\frac{\max(|\delta_i| - \delta_{\max}, 0)}{\delta_{\max}}\right)^2\right].$$

5.5 Smoothness Cost

The smoothness term penalizes second differences in both sampled path points and B-spline controls:

$$J_R = \text{mean}\left[\|\mathbf{p}_{i+2} - 2\mathbf{p}_{i+1} + \mathbf{p}_i\|_2^2\right] + \text{mean}\left[\|\mathbf{c}_{j+2} - 2\mathbf{c}_{j+1} + \mathbf{c}_j\|_2^2\right].$$

6 Optimization Procedure

The optimization loop in `optimize_terrain_aware_path` proceeds as follows:

1. Validate the seed path shape $(N, 2)$ and finite numeric values.
2. Initialize B-spline control points by arc-length sampling the seed path, unless explicit initial controls are provided.
3. Build a clamped B-spline basis with `num_samples` rows.
4. Freeze the start and goal control points.
5. Create an Adam optimizer over the interior controls.
6. Sample the initial path and compute raw initial costs.
7. Build cost normalizers from the initial raw costs if normalization is enabled.
8. For each iteration:
   (a) concatenate fixed start, trainable interior controls, and fixed goal;
   (b) sample the B-spline path;
   (c) query the height field and gradients;
   (d) compute yaw, curvature, slope, steering, throttle, length, and smoothness;
   (e) form the weighted objective;
   (f) backpropagate through the differentiable path and terrain queries;
   (g) update interior controls using Adam;

(h) clamp interior controls to `nemo.field.bounds`, optionally with `bounds_margin`;

(i) re-evaluate the candidate path and retain the best observed interior controls.

9. Reconstruct the best path, query final heights, and return diagnostics plus trajectory arrays.

In either gradient mode, gradients flow through the height-field evaluations with respect to the query coordinates. This allows the optimizer to move path controls in directions that reduce terrain-induced costs. In finite-difference mode, this happens through the shifted $h(x, y)$ calls. In autograd mode, it happens through the local derivative of h at each sampled path point.

7 Diagnostics and Outputs

The result object contains:

- `initial_path_xy`, `optimized_path_xy`: sampled 2D paths.
- `initial_path_xyz`, `optimized_path_xyz`: terrain-grounded 3D paths using $z = h(x, y)$.
- `control_points_initial`, `control_points_optimized`: B-spline controls.
- `initial_diagnostics`, `optimized_diagnostics`: scalar cost and feasibility metrics.
- `cost_history`: candidate objective value per iteration.
- `trajectory`: arrays for $x, y, z, \psi, \kappa, \alpha, \beta^{along}, \beta^{cross}, v, \delta, u^{thr}$.

Diagnostics include total cost, each raw cost term, 3D length, mean/max total slope, mean/max uphill and downhill angles, mean/max cross-track angle, mean/max throttle magnitude, and mean/max steering angle.

8 Relationship to Dense Reference Trajectories

The terrain-aware planner returns an optimized terrain-grounded path. For AirSim-style tracking, `nemo/airsim_trajectory.py` can resample that 3D path into a dense reference trajectory. That post-processing computes:

$$\psi_i = \operatorname{atan2}\left(\frac{dy}{ds}, \frac{dx}{ds}\right),$$

$$\kappa_i = \frac{x_i' y_i'' - y_i' x_i''}{(x_i'^2 + y_i'^2)^{3/2}},$$

$$\theta_i = \operatorname{atan2}(\Delta z_i, \Delta s_{xy,i}),$$

where θ_i is path pitch. The speed profile is limited by nominal speed, curvature, yaw rate, slope, and acceleration constraints:

$$v_{curve,i} = \sqrt{\frac{a_{\max}^{lat}}{|\kappa_i|}}, \qquad v_{yaw,i} = \frac{\omega_{\max}}{|\kappa_i|},$$

$$v_{slope,i} = \frac{v_{nom}}{1 + k_{slope} |\tan(\theta_i)|}.$$

The reference trajectory stores states

$$[x, y, z, \psi, \theta, \phi, v, \dot{\psi}]$$

and controls

$$[v_{cmd}, \dot{\psi}_{cmd}, a_{cmd}].$$

9 Implementation Map

Concept	Implementation
Planner config and result dataclasses	`nemo/terrain_aware_planner.py`
B-spline initialization and sampling	`initialize_control_points`, `bspline_basis`, `sample_path`
Height and gradient query	`path_xyz`, `terrain_grad`, `finite_difference_grad`, `autograd_grad`
Trajectory quantities	`trajectory_from_path`
Objective terms	`compute_raw_costs`, `_weighted_total`
Diagnostics	`make_diagnostics`, `_trajectory_to_numpy`
Crater-entry driver	`scripts/terrain_aware_planner.py`
Dense AirSim reference generation	`nemo/airsim_trajectory.py`

10 Practical Interpretation

The planner is best understood as a differentiable post-processor for a geometric seed path. A* or manual waypoints provide a globally reasonable route. The B-spline optimizer then uses the NEMo height field to locally trade distance against terrain feasibility. If the seed crosses a steep hill, the uphill/downhill/throttle terms push the path sideways. If the path turns too tightly, the steering and smoothness terms spread the turn. If the terrain rolls sharply across the vehicle body, the cross-track term discourages traversing side-slopes.

The output is therefore not simply the shortest path over the height field. It is the lowest-cost compromise between length, terrain slope, actuator proxies, and smoothness under fixed start/goal constraints.